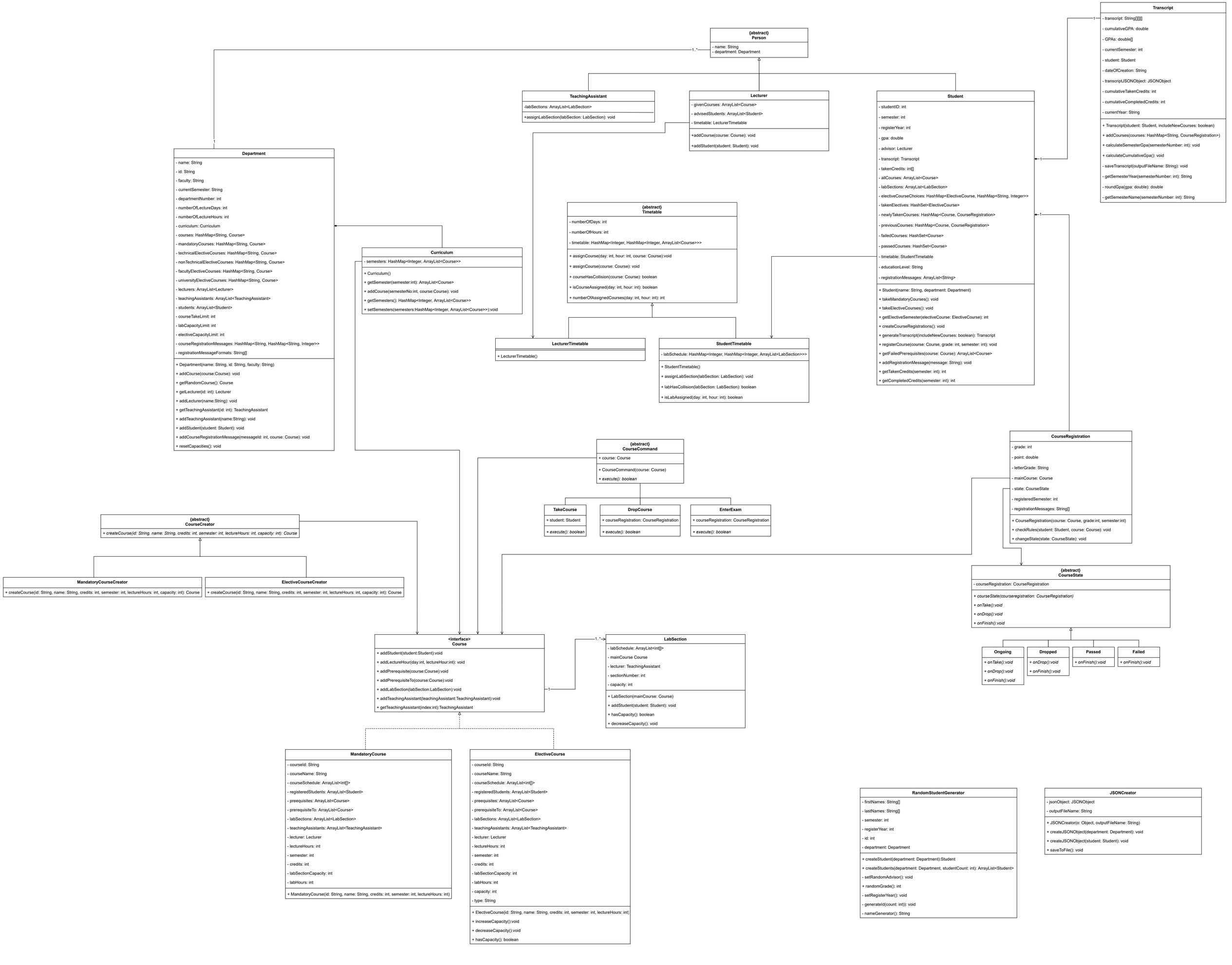

{abstract}
Person
- name: String
- department: Department
1..*
1
Department
- name: String
- id: String
- faculty: String
- currentSemester: String
- departmentNumber: int
- numberOfLectureDays: int
- numberOfLectureHours: int
- curriculum: Curriculum
- courses: HashMap<String, Course>
- mandatoryCourses: HashMap<String, Course>
- technicalElectiveCourses: HashMap<String, Course>
- nonTechnicalElectiveCourses: HashMap<String, Course>
- facultyElectiveCourses: HashMap<String, Course>
- universityElectiveCourses: HashMap<String, Course>
- lecturers: ArrayList<Lecturer>
- teachingAssistants: ArrayList<TeachingAssistant>
- students: ArrayList<Student>
- courseTakeLimit: int
- labCapacityLimit: int
- electiveCapacityLimit: int
- courseRegistrationMessages: HashMap<String, HashMap<String, Integer>>
- registrationMessageFormats: String[]
+ Department(name: String, id: String, faculty: String)
+ addCourse(course:Course): void
+ getRandomCourse(): Course
+ getLecturer(id: int): Lecturer
+ addLecturer(name:String): void
+ getTeachingAssistant(id: int): TeachingAssistant
+ addTeachingAssistant(name:String): void
+ addStudent(student: Student): void
+ addCourseRegistrationMessage(messageId: int, course: Course): void
+ resetCapacities(): void
TeachingAssistant
-labSections: ArrayList<LabSection>
+assignLabSection(labSection: LabSection): void
Lecturer
- givenCourses: ArrayList<Course>
- advisedStudents: ArrayList<Student>
- timetable: LecturerTimetable
+addCourse(course: Course): void
+addStudent(student: Student): void
Student
- studentID: int
- semester: int
- registerYear: int
- gpa: double
- advisor: Lecturer
- transcript: Transcript
- takenCredits: int[]
- allCourses: ArrayList<Course>
- labSections: ArrayList<LabSection>
- electiveCourseChoices: HashMap<ElectiveCourse, HashMap<String, Integer>>
- takenElectives: HashSet<ElectiveCourse>
- newlyTakenCourses: HashMap<Course, CourseRegistration>
- previousCourses: HashMap<Course, CourseRegistration>
- failedCourses: HashSet<Course>
- passedCourses: HashSet<Course>
- timetable: StudentTimetable
- educationLevel: String
- registrationMessages: ArrayList<String>
+ Student(name: String, department: Department)
+ takeMandatoryCourses(): void
+ takeElectiveCourses(): void
+ getElectiveSemester(electiveCourse: ElectiveCourse): int
+ createCourseRegistrations(): void
+ generateTranscript(includeNewCourses: boolean): Transcript
+ registerCourse(course: Course, grade: int, semester: int): void
+ getFailedPrerequisites(course: Course): ArrayList<Course>
+ addRegistrationMessage(message: String): void
+ getTakenCredits(semester: int): int
+ getCompletedCredits(semester: int): int
Transcript
- transcript: String[][][]
- cumulativeGPA: double
- GPAs: double[]
- currentSemester: int
- student: Student
- dateOfCreation: String
- transcriptJSONObject: JSONObject
- cumulativeTakenCredits: int
- cumulativeCompletedCredits: int
- currentYear: String
+ Transcript(student: Student, includeNewCourses: boolean)
+ addCourses(courses: HashMap<String, CourseRegistration>)
+ calculateSemesterGpa(semesterNumber: int): void
+ calculateCumulativeGpa(): void
- saveTranscript(outputFileName: String): void
- getSemesterYear(semesterNumber: int): String
- roundGpa(gpa: double): double
- getSemesterName(semesterNumber: int): String
{abstract}
Timetable
- numberOfDays: int
- numberOfHours: int
- timetable: HashMap<Integer, HashMap<Integer, ArrayList<Course>>>
+ assignCourse(day: int, hour: int, course: Course):void
+ assignCourse(course: Course): void
+ courseHasCollision(course: Course): boolean
+ isCourseAssigned(day: int, hour: int): boolean
+ numberOfAssignedCourses(day: int, hour: int): int
Curriculum
- semesters: HashMap<Integer, ArrayList<Course>>
+ Curriculum()
+ getSemester(semester:int): ArrayList<Course>
+ addCourse(semesterNo:int, course:Course): void
+ getSemesters(): HashMap<Integer, ArrayList<Course>>
+ setSemesters(semesters:HashMap<Integer, ArrayList<Course>>):void
LecturerTimetable
+ LecturerTimetable()
StudentTimetable
- labSchedule: HashMap<Integer, HashMap<Integer, ArrayList<LabSection>>>
+ StudentTimetable()
+ assignLabSection(labSection: LabSection): void
+ labHasCollision(labSection: LabSection): boolean
+ isLabAssigned(day: int, hour: int): boolean
{abstract}
CourseCommand
+ course: Course
+ CourseCommand(course: Course)
+ execute(): boolean
TakeCourse
+ student: Student
+ execute(): boolean
DropCourse
+ courseRegistration: CourseRegistration
+ execute(): boolean
EnterExam
+ courseRegistration: CourseRegistration
+ execute(): boolean
CourseRegistration
- grade: int
- point: double
- letterGrade: String
- mainCourse: Course
- state: CourseState
- registeredSemester: int
- registrationMessages: String[]
+ CourseRegistration(course: Course, grade:int, semester:int)
+ checkRules(student: Student, course: Course): void
+ changeState(state: CourseState): void
{abstract}
CourseState
- courseRegistration: CourseRegistration
+ courseState(courseregistration: CourseRegistration)
+ onTake():void
+ onDrop():void
+ onFinish():void
Ongoing
+ onTake():void
+ onDrop():void
+ onFinish():void
Dropped
+ onDrop():void
+ onFinish():void
Passed
+ onFinish():void
Failed
+ onFinish():void
{abstract}
CourseCreator
+ createCourse(id: String, name: String, credits: int, semester: int, lectureHours: int, capacity: int): Course
MandatoryCourseCreator
+ createCourse(id: String, name: String, credits: int, semester: int, lectureHours: int, capacity: int): Course
ElectiveCourseCreator
+ createCourse(id: String, name: String, credits: int, semester: int, lectureHours: int, capacity: int): Course
<interface>
Course
+ addStudent(student:Student):void
+ addLectureHour(day:int, lectureHour:int): void
+ addPrerequisite(course:Course):void
+ addPrerequisiteTo(course:Course):void
+ addLabSection(labSection:LabSection):void
+ addTeachingAssistant(teachingAssistant:TeachingAssistant):void
+ getTeachingAssistant(index:int):TeachingAssistant
1..*
1
LabSection
- labSchedule: ArrayList<int[]>
- mainCourse Course
- lecturer: TeachingAssistant
- sectionNumber: int
- capacity: int
+ LabSection(mainCourse: Course)
+ addStudent(student: Student): void
+ hasCapacity(): boolean
+ decreaseCapacity(): void
MandatoryCourse
- courseId: String
- courseName: String
- courseSchedule: ArrayList<int[]>
- registeredStudents: ArrayList<Student>
- preequisites: ArrayList<Course>
- prerequisiteTo: ArrayList<Course>
- labSections: ArrayList<LabSection>
- teachingAssistants: ArrayList<TeachingAssistant>
- lecturer: Lecturer
- lectureHours: int
- semester: int
- credits: int
- labSectionCapacity: int
- labHours: int
+ MandatoryCourse(id: String, name: String, credits: int, semester: int, lectureHours: int)
ElectiveCourse
- courseId: String
- courseName: String
- courseSchedule: ArrayList<int[]>
- registeredStudents: ArrayList<Student>
- preequisites: ArrayList<Course>
- prerequisiteTo: ArrayList<Course>
- labSections: ArrayList<LabSection>
- teachingAssistants: ArrayList<TeachingAssistant>
- lecturer: Lecturer
- lectureHours: int
- semester: int
- credits: int
- labSectionCapacity: int
- labHours: int
- capacity: int
- type: String
+ ElectiveCourse(id: String, name: String, credits: int, semester: int, lectureHours: int)
+ increaseCapacity():void
+ decreaseCapacity():void
+ hasCapacity(): boolean
RandomStudentGenerator
- firstNames: String[]
- lastNames: String[]
- semester: int
- registerYear: int
- id: int
- department: Department
+ createStudent(department: Department):Student
+ createStudents(department: Department, studentCount: int): ArrayList<Student>
- setRandomAdvisor(): void
+ randomGrade(): int
- setRegisterYear(): void
- generateId(count: int)): void
- nameGenerator(): String
JSONCreator
- jsonObject: JSONObject
- outputFileName: String
+ JSONCreator(o: Object, outputFileName: String)
+ createJSONObject(department: Department): void
+ createJSONObject(student: Student): void
+ saveToFile(): void